

# Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3084 Computación Paralela y Distribuida - Sección 20, Semestre II 2026



## Proyecto #1

### Integrantes del grupo

Roberto Barreda #23354

Nina Najera #231088

José Antón #221041

Guatemala, 2 de septiembre de 2026

# Índice

|                                                               |          |
|---------------------------------------------------------------|----------|
| <b>1. Introducción</b>                                        | <b>1</b> |
| <b>2. Antecedentes</b>                                        | <b>1</b> |
| 2.1. OpenMP . . . . .                                         | 1        |
| 2.2. Simulación de N cuerpos . . . . .                        | 1        |
| 2.3. Método PCAM . . . . .                                    | 2        |
| <b>3. Descripción y funcionamiento del programa</b>           | <b>2</b> |
| 3.1. Objetivo de la simulación . . . . .                      | 2        |
| 3.2. Cumplimiento de los requisitos del screensaver . . . . . | 2        |
| 3.3. Escenario de colisión . . . . .                          | 2        |
| 3.4. Escenario de galaxia . . . . .                           | 4        |
| 3.5. Escenario de nube . . . . .                              | 4        |
| 3.6. Estructura de los datos . . . . .                        | 6        |
| 3.7. Renderizado y panel de métricas . . . . .                | 6        |
| <b>4. Estrategia de paralelización</b>                        | <b>7</b> |
| 4.1. Partición . . . . .                                      | 7        |
| 4.2. Comunicación . . . . .                                   | 7        |
| 4.3. Aglomeración . . . . .                                   | 7        |
| 4.4. Mapeo . . . . .                                          | 7        |
| 4.5. Sincronización . . . . .                                 | 8        |
| <b>5. Programación defensiva y manejo de recursos</b>         | <b>8</b> |
| <b>6. Compilación y ejecución</b>                             | <b>8</b> |
| <b>7. Resultados y análisis</b>                               | <b>9</b> |
| 7.1. Resultados de la ejecución visual . . . . .              | 9        |
| 7.2. Configuración de la prueba de rendimiento . . . . .      | 9        |
| 7.3. Análisis del speedup y la eficiencia . . . . .           | 9        |

|                                            |    |
|--------------------------------------------|----|
| 8. Conclusiones                            | 11 |
| 9. Recomendaciones                         | 12 |
| Referencias                                | 12 |
| A. Anexo 1. Diagrama de flujo del programa | 12 |
| B. Anexo 2. Catálogo de funciones          | 15 |
| C. Anexo 3. Bitácora de pruebas            | 19 |

# 1. Introducción

El proyecto consiste en el desarrollo de un screensaver basado en una simulación gravitacional de N cuerpos. Cada cuerpo posee posición, velocidad, aceleración y masa. Durante cada paso de la simulación, los cuerpos se atraen entre sí y generan escenas con movimiento continuo.

La solución fue desarrollada en C++ y utiliza SDL2 para mostrar la simulación en pantalla. También se implementó una versión paralela con OpenMP. El propósito de la paralelización es distribuir entre varios hilos las operaciones que requieren mayor tiempo de procesamiento y mejorar el desempeño cuando aumenta la cantidad de cuerpos.

El programa incluye una versión secuencial y una versión paralela construidas a partir de los mismos archivos fuente. Además, permite configurar la cantidad de cuerpos, el escenario, el tamaño del canvas, la cantidad de hilos, la semilla pseudoaleatoria, los parámetros físicos y la política de reparto de OpenMP.

## 2. Antecedentes

### 2.1. OpenMP

OpenMP es una interfaz de programación paralela orientada a sistemas de memoria compartida. Permite distribuir trabajo entre varios hilos mediante directivas que se agregan al código en C y C++. Esto facilita transformar de manera gradual una solución secuencial en una solución paralela [1].

En este proyecto se utilizan ciclos paralelos para el cálculo de aceleraciones, la integración del movimiento, el desvanecimiento del framebuffer y otras operaciones que pueden dividirse entre varios hilos.

### 2.2. Simulación de N cuerpos

Una simulación de N cuerpos representa la interacción entre varios objetos que se atraen mediante gravedad. En la solución directa implementada, cada cuerpo considera la influencia de todos los demás. Por esta razón, la cantidad de operaciones crece de forma cuadrática al aumentar N [3].

La aceleración utilizada en la simulación se expresa como:

$$\vec{a}_i = G \sum_j m_j \frac{\vec{r}_j - \vec{r}_i}{[r_{ij}^2 + \varepsilon^2]^{3/2}} \quad (1)$$

El término  $\varepsilon$  corresponde al suavizado de Plummer. Su función es evitar valores demasiado grandes cuando dos cuerpos se encuentran muy cerca.

Después del cálculo de aceleración se actualizan la velocidad y la posición mediante Euler semiimplícito:

$$\vec{v}_i \leftarrow \vec{v}_i + \vec{a}_i \Delta t \quad (2)$$

$$\vec{r}_i \leftarrow \vec{r}_i + \vec{v}_i \Delta t \quad (3)$$

### 2.3. Método PCAM

PCAM divide el diseño de una solución paralela en cuatro etapas: partición, comunicación, aglomeración y mapeo [2]. Este método es útil para identificar qué trabajo puede ejecutarse al mismo tiempo y cómo debe distribuirse entre los recursos disponibles.

La simulación se adapta a este enfoque porque el cálculo de aceleración de un cuerpo puede realizarse de forma independiente mientras las posiciones de los demás cuerpos permanezcan sin cambios durante esa fase.

## 3. Descripción y funcionamiento del programa

### 3.1. Objetivo de la simulación

El programa representa cuerpos que se atraen por gravedad y se desplazan de forma continua dentro de una escena. El movimiento se obtiene directamente de los cálculos físicos y no de trayectorias dibujadas previamente.

Se implementaron tres escenarios. El modo colisión crea dos galaxias que se aproximan entre sí. El modo galaxia crea un solo disco en rotación. El modo nube distribuye los cuerpos de forma pseudoaleatoria y permite observar cómo la gravedad modifica su distribución.

### 3.2. Cumplimiento de los requisitos del screensaver

### 3.3. Escenario de colisión

En el modo colisión se crean dos discos galácticos. Cada uno posee un núcleo con mayor masa y un conjunto de cuerpos distribuidos alrededor de él. Las dos galaxias se generan con velocidades opuestas y con un desplazamiento que permite observar deformaciones durante la interacción.

Cuadro 1: Cumplimiento de los requisitos principales

| Requisito                | Implementación                                                                                                                        |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Cantidad N de elementos  | La cantidad de cuerpos se recibe con la bandera <b>-n</b> . El rango permitido es de 2 a 200000 cuerpos.                              |
| Varios colores           | Cada cuerpo utiliza una rampa de color combinada con un desplazamiento pseudoaleatorio generado a partir de la semilla.               |
| Tamaño mínimo del canvas | El ancho mínimo es 640 y el alto mínimo es 480. El valor predeterminado es 1000 por 700.                                              |
| Movimiento               | La velocidad y la posición de los cuerpos se actualizan en cada paso de la simulación.                                                |
| Física y trigonometría   | El programa utiliza gravitación, suavizado de Plummer, seno, coseno y velocidad tangencial para la creación de los discos galácticos. |
| FPS                      | El programa calcula y muestra los FPS junto con el tiempo de física y el tiempo de renderizado.                                       |

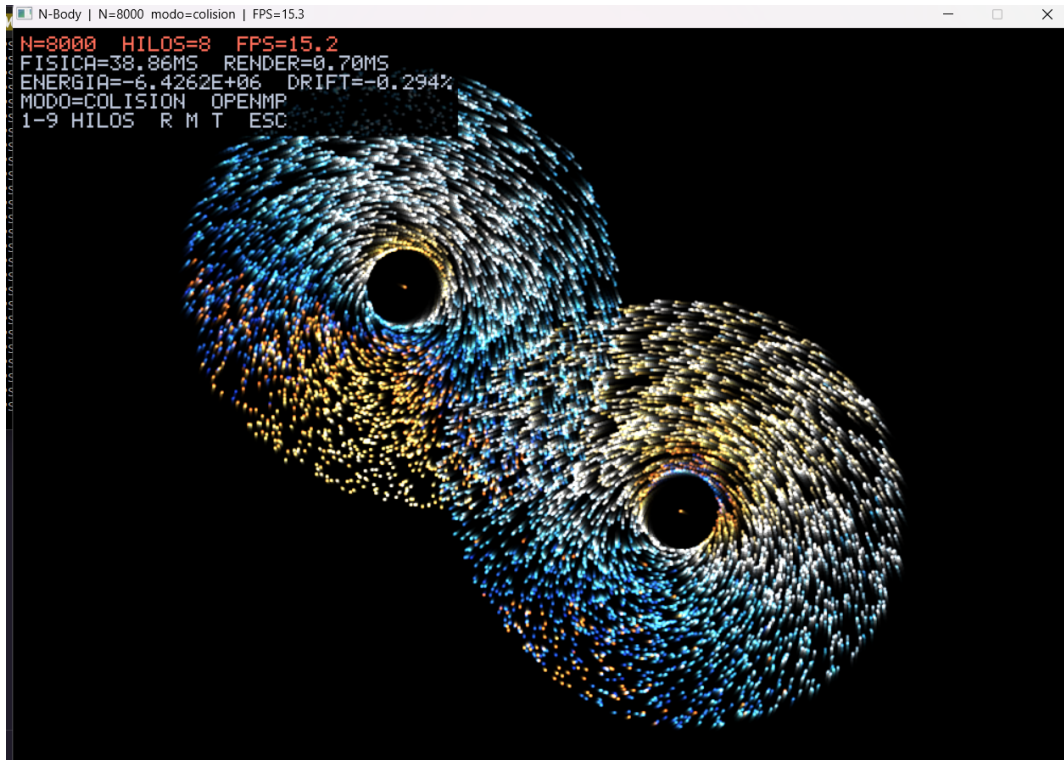


Figura 1: Escenario de colisión con 8000 cuerpos y 8 hilos

### 3.4. Escenario de galaxia

El modo galaxia genera un solo disco alrededor de un núcleo. Las estrellas reciben una posición radial y una velocidad tangencial calculada a partir de la masa encerrada dentro de su radio. Esto produce un patrón de rotación alrededor del centro.

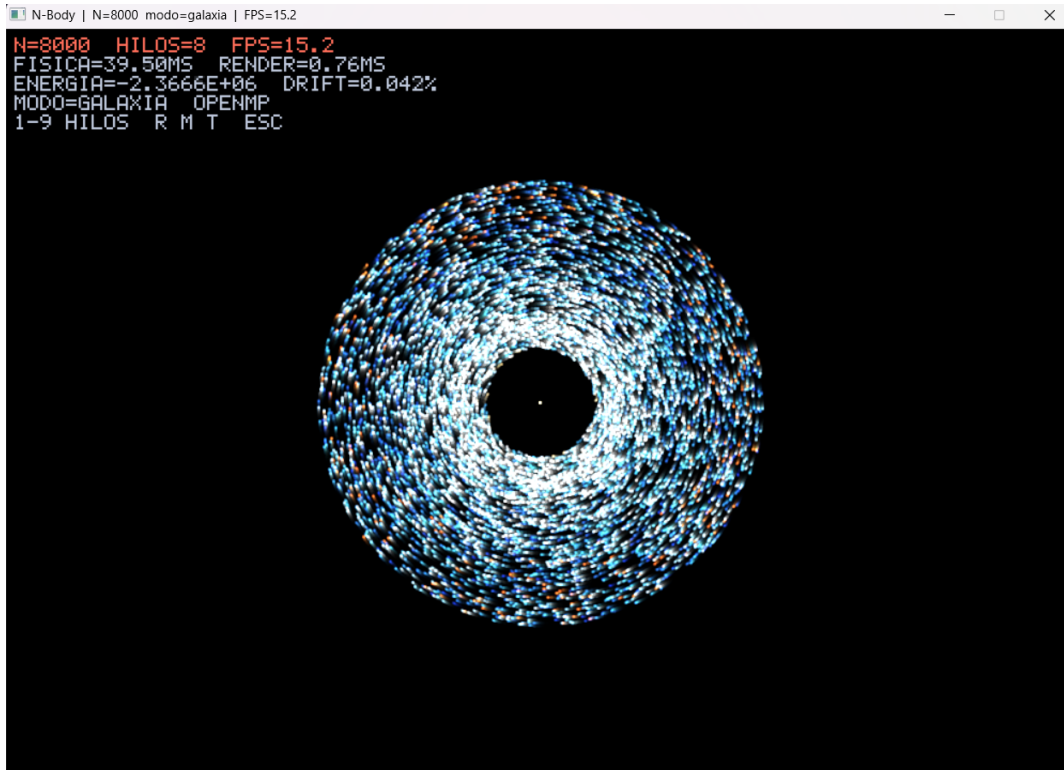


Figura 2: Escenario de galaxia con 8000 cuerpos y 8 hilos

### 3.5. Escenario de nube

El modo nube distribuye cuerpos con masas iguales dentro del canvas. Las velocidades iniciales son pequeñas y la gravedad modifica progresivamente la forma del conjunto.

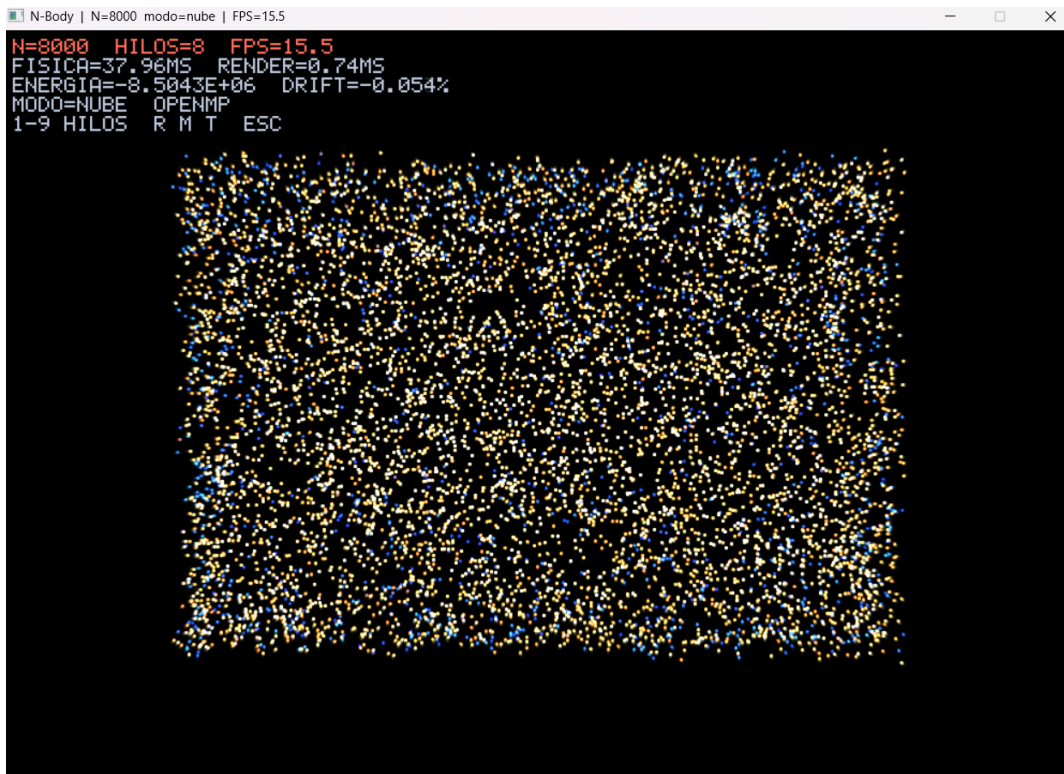


Figura 3: Escenario de nube con 8000 cuerpos y 8 hilos



### 3.6. Estructura de los datos

Las posiciones, velocidades, aceleraciones, masas y variaciones de color se almacenan en arreglos separados. Esta organización permite recorrer de manera continua un mismo atributo para todos los cuerpos y facilita el acceso a memoria durante los ciclos principales.

La generación pseudoaleatoria utiliza Xorshift32. El estado de este generador no depende de una variable global compartida. La semilla permite reproducir la misma configuración inicial cuando se utiliza el mismo valor.

### 3.7. Renderizado y panel de métricas

SDL2 se utiliza para crear la ventana, el renderer y la textura que presenta el framebuffer en pantalla [4]. El framebuffer utiliza el formato ARGB8888.

Cuando las estelas están activadas, el contenido anterior no se elimina de inmediato. Los píxeles disminuyen su intensidad y dejan un rastro del movimiento. Los cuerpos se dibujan mediante pequeños destellos con suma de brillo.

El panel superior muestra la cantidad de cuerpos, la cantidad de hilos, los FPS, el tiempo de física, el tiempo de renderizado, la energía, la variación de energía y el modo actual. También indica si la aplicación fue compilada con OpenMP.

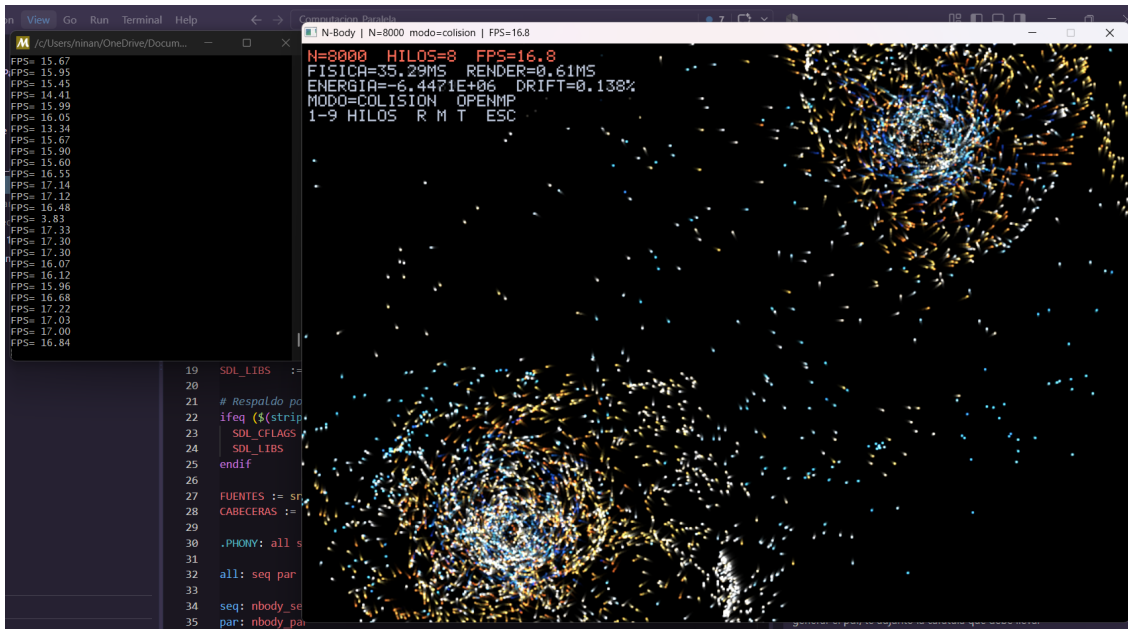


Figura 4: Ejecución de la versión paralela y despliegue de métricas

## 4. Estrategia de paralelización

### 4.1. Partición

El cálculo de aceleraciones es la parte principal de la simulación. Para cada cuerpo se recorren todos los demás cuerpos. La iteración externa se divide entre varios hilos porque cada iteración escribe únicamente la aceleración correspondiente a su propio índice.

La integración también se divide entre hilos. Cada iteración actualiza la velocidad y la posición de un solo cuerpo. El desvanecimiento del framebuffer se paraleliza por píxel porque cada posición de la imagen puede procesarse de forma independiente.

El dibujo de cuerpos requiere una estrategia diferente. Dos cuerpos pueden intentar modificar el mismo píxel. Para evitar esta condición de carrera, el framebuffer se divide en bandas horizontales. Cada hilo escribe únicamente en las filas que le pertenecen.

### 4.2. Comunicación

Durante el cálculo de aceleraciones, las posiciones y las masas son compartidas únicamente para lectura. Cada hilo utiliza acumuladores locales para calcular la aceleración de los cuerpos que le corresponden.

Para los cálculos de energía se utilizan reducciones de OpenMP. Cada hilo mantiene un resultado parcial y OpenMP combina los resultados al finalizar la región paralela.

### 4.3. Aglomeración

El cálculo de aceleraciones utiliza `schedule runtime`. Esto permite elegir la política de reparto desde la línea de comandos con `static`, `dynamic` o `guided`.

Para la bitácora final de rendimiento se utilizó la política `dynamic`. De esta forma, las iteraciones se entregan a los hilos conforme estos terminan trabajo previo. La misma política y los mismos parámetros se mantuvieron en todas las configuraciones de la prueba para que la comparación entre cantidades de hilos fuera consistente.

### 4.4. Mapeo

OpenMP distribuye las iteraciones entre los hilos disponibles. La cantidad de hilos puede establecerse con la bandera `-t`. El valor cero permite que OpenMP seleccione la cantidad disponible.

Durante la ejecución gráfica también es posible cambiar la cantidad de hilos con las teclas del 1 al 9. Esto permite observar el efecto de la cantidad de hilos sin reiniciar el programa.

## 4.5. Sincronización

El programa separa el cálculo de aceleraciones y la integración en dos fases. Al terminar el ciclo paralelo de aceleraciones existe una barrera implícita de OpenMP. La integración comienza únicamente cuando todas las aceleraciones ya fueron calculadas.

Esta separación evita que un hilo modifique una posición mientras otro hilo todavía necesita leer el valor anterior. En el renderizado, la división por bandas evita la escritura concurrente sobre los mismos píxeles sin utilizar secciones críticas ni operaciones atómicas.

## 5. Programación defensiva y manejo de recursos

El programa valida los argumentos antes de inicializar SDL2 o reservar los recursos gráficos. Los valores numéricos se comprueban para asegurar que tengan el tipo correcto y que se encuentren dentro de los rangos permitidos.

La cantidad de cuerpos, las dimensiones del canvas, la cantidad de hilos, la gravedad, el suavizado y el paso de tiempo tienen límites definidos. Las opciones desconocidas generan un mensaje de error y terminan la ejecución de forma controlada.

El suavizado tiene un mínimo de 0.001. Además, el cálculo de aceleraciones mantiene un valor mínimo interno para evitar divisiones que puedan generar valores no numéricos.

La reserva del sistema y del framebuffer se encuentra dentro de un bloque que captura errores de memoria insuficiente. Los recursos de SDL2 se agrupan en una estructura que destruye la textura, el renderer y la ventana al terminar la ejecución.

Antes de convertir posiciones a coordenadas enteras, el renderizado verifica que sean valores finitos y representables. Los cuerpos que se encuentran demasiado lejos del canvas no se dibujan.

## 6. Compilación y ejecución

El proyecto utiliza C++17, SDL2 y OpenMP. El archivo `Makefile` genera las dos versiones a partir de los mismos archivos fuente.

Cuadro 2: Versiones generadas por el proyecto

| Ejecutable | Descripción                              |
|------------|------------------------------------------|
| nbody_seq  | Versión secuencial compilada sin OpenMP. |
| nbody_par  | Versión paralela compilada con OpenMP.   |

Para compilar ambas versiones se utiliza:

`make`

Un ejemplo de ejecución paralela es:

```
./nbody_par.exe -n 8000 -m collision -t 8
```

El modo de benchmark ejecuta la física y el renderizado en memoria sin abrir una ventana. De esta manera se evita incluir el costo de presentación de SDL2 en las mediciones.

```
./nbody_par.exe --bench -n 8000 -t 8 -f 50
```

## 7. Resultados y análisis

### 7.1. Resultados de la ejecución visual

Las capturas de los tres escenarios muestran una ejecución con 8000 cuerpos y 8 hilos. En las imágenes se observan valores cercanos a 15 y 17 FPS. Estos valores corresponden al modo interactivo y no deben compararse directamente con el benchmark sin ventana, ya que la ejecución visual también realiza actualización de textura, presentación y manejo de eventos.

En las capturas también se observa que el tiempo de física es considerablemente mayor que el tiempo de renderizado. Esto coincide con la estructura del algoritmo, donde el cálculo de interacciones entre cuerpos concentra la mayor parte del trabajo.

### 7.2. Configuración de la prueba de rendimiento

La prueba final se ejecutó en modo colisión con la política `dynamic`. Se evaluaron dos tamaños de problema, 4000 y 8000 cuerpos. Para cada tamaño se midió la versión secuencial y la versión paralela con 1, 2, 4 y 8 hilos.

Cada configuración se ejecutó 10 veces y cada repetición midió 20 frames después del calentamiento. En total se obtuvieron 100 mediciones. Los valores presentados en la siguiente tabla corresponden al promedio de las 10 repeticiones de cada configuración.

El speedup de la fila secuencial es 1.00x por definición, ya que es la referencia contra la que se comparan las demás configuraciones del mismo tamaño de problema.

### 7.3. Análisis del speedup y la eficiencia

El speedup compara el tiempo de la versión secuencial con el tiempo de la versión paralela:

$$S_p = \frac{T_s}{T_p} \quad (4)$$

La eficiencia relaciona el speedup con la cantidad de hilos utilizados:

Cuadro 3: Resultados promedio del benchmark

| N    | Versión    | Hilos | ms/frame | FPS   | Speedup | Eficiencia |
|------|------------|-------|----------|-------|---------|------------|
| 4000 | Secuencial | 1     | 15.149   | 66.2  | 1.00x   | –          |
| 4000 | Paralela   | 1     | 29.892   | 33.5  | 0.50x   | 50 %       |
| 4000 | Paralela   | 2     | 16.104   | 62.4  | 0.93x   | 46 %       |
| 4000 | Paralela   | 4     | 9.855    | 101.9 | 1.52x   | 38 %       |
| 4000 | Paralela   | 8     | 8.519    | 117.5 | 1.75x   | 22 %       |
| 8000 | Secuencial | 1     | 58.745   | 17.0  | 1.00x   | –          |
| 8000 | Paralela   | 1     | 131.675  | 7.7   | 0.45x   | 45 %       |
| 8000 | Paralela   | 2     | 71.746   | 14.0  | 0.82x   | 41 %       |
| 8000 | Paralela   | 4     | 44.808   | 22.6  | 1.33x   | 33 %       |
| 8000 | Paralela   | 8     | 42.649   | 23.6  | 1.38x   | 17 %       |

$$E_p = \frac{S_p}{p} \quad (5)$$

Con 4000 cuerpos, la versión paralela comienza a superar a la secuencial a partir de 4 hilos. Con 4 hilos alcanza un speedup promedio de 1.52 y con 8 hilos llega a 1.75. Esto reduce el tiempo promedio por frame de 15.149 ms en la versión secuencial a 8.519 ms con 8 hilos.

Con 8000 cuerpos ocurre un comportamiento similar. La versión paralela con 1 y 2 hilos resulta más lenta que la versión secuencial, mientras que con 4 y 8 hilos sí se obtiene aceleración. El mayor speedup medido fue de 1.38 con 8 hilos.

### Por qué la versión paralela con un hilo es más lenta

La versión paralela con un solo hilo presenta un tiempo mayor que la versión secuencial. La causa no es el costo de crear y administrar regiones paralelas: entrar a una región paralela con un solo hilo cuesta del orden de microsegundos, mientras que la diferencia medida es de aproximadamente 73 ms por frame con 8000 cuerpos, es decir unas mil veces mayor.

La causa real es que al compilar con `-fopenmp` el compilador traslada el cuerpo de la región paralela a una función generada internamente, y en ese contexto el vectorizador de GCC deja de procesar el ciclo interno. Esto se comprueba con la opción de diagnóstico `-fopt-info-vec`, que reporta:

```
sin -fopenmp: loop vectorized using 32 byte vectors and unroll factor 8
con -fopenmp: not vectorized: unsupported control flow in loop
```

Sin vectorización cada hilo procesa un par de cuerpos por instrucción en lugar de ocho, lo que explica que el rendimiento por hilo se reduzca aproximadamente a la mitad. Por esta razón

la versión paralela necesita alrededor de tres hilos únicamente para recuperar el rendimiento que pierde al compilarse, y comienza a superar a la versión secuencial a partir de cuatro hilos.

Se evaluaron cuatro alternativas para recuperar la vectorización: la directiva `omp simd` sobre el ciclo interno, extraer el ciclo a una función independiente, devolver el resultado por valor en lugar de por referencia, y aplicar bloqueo de registros procesando varios cuerpos a la vez. Las cuatro empeoraron el tiempo absoluto respecto a la forma actual, por lo que se conservó la versión más rápida de las medidas.

## Comportamiento de la eficiencia

La eficiencia disminuye al aumentar la cantidad de hilos. Para 4000 cuerpos pasa de aproximadamente 50 por ciento con un hilo paralelo a 22 por ciento con 8 hilos. Para 8000 cuerpos disminuye de aproximadamente 45 por ciento a 17 por ciento. Esto indica que el speedup no crece de forma proporcional al número de hilos debido al costo de administración, sincronización y otras partes del programa que no se aceleran en la misma proporción.

Los resultados también muestran que 4 hilos ofrecen un mejor balance entre aceleración y eficiencia. En 4000 cuerpos se obtiene un speedup de 1.52 con una eficiencia de 38 por ciento. En 8000 cuerpos se obtiene un speedup de 1.33 con una eficiencia de 33 por ciento. El uso de 8 hilos entrega el mayor speedup, pero con una disminución considerable de eficiencia.

## 8. Conclusiones

La simulación de  $N$  cuerpos es adecuada para aplicar paralelización porque el cálculo de aceleraciones contiene una gran cantidad de operaciones independientes. Cada iteración externa puede asignarse a un hilo distinto sin modificar las posiciones que los demás hilos están leyendo.

La separación entre el cálculo de aceleraciones y la integración permite mantener un estado consistente durante cada paso. La barrera implícita de OpenMP asegura que todas las aceleraciones estén listas antes de actualizar posiciones y velocidades.

La estrategia de dividir el framebuffer en bandas horizontales permite paralelizar el dibujo sin producir condiciones de carrera sobre los píxeles. Cada hilo trabaja únicamente dentro de su propia región de la imagen.

Las 100 mediciones realizadas muestran que la paralelización sí mejora el rendimiento cuando se utilizan suficientes hilos. Para 4000 cuerpos el mejor resultado fue de 117.5 FPS y un speedup de 1.75 con 8 hilos. Para 8000 cuerpos el mejor resultado fue de 23.6 FPS y un speedup de 1.38 también con 8 hilos.

La versión paralela con 1 y 2 hilos no supera a la versión secuencial en las pruebas realizadas. Esto se debe a la pérdida de vectorización que introduce la compilación con OpenMP, y demuestra que utilizar más hilos no garantiza por sí solo una mejora de rendimiento.

La eficiencia disminuye conforme aumenta la cantidad de hilos. Por esta razón, 4 hilos presentan un mejor balance entre aceleración y aprovechamiento de los recursos, mientras que 8 hilos permiten obtener el mayor speedup medido.

## 9. Recomendaciones

Se recomienda realizar las mediciones de rendimiento con la menor cantidad posible de aplicaciones adicionales en ejecución. Esto ayuda a reducir variaciones provocadas por otros procesos del sistema.

También se recomienda utilizar la misma semilla, el mismo escenario, la misma cantidad de frames y los mismos parámetros físicos cuando se comparan distintas cantidades de hilos. De esta manera, las configuraciones realizan una carga de trabajo equivalente.

Para comparar el desempeño se recomienda utilizar el modo `--bench`, ya que mide la física y el renderizado sin incluir el costo de mostrar la ventana. La ejecución visual puede utilizarse como evidencia del funcionamiento y para observar los FPS desde el punto de vista del usuario.

Si se busca el mayor rendimiento de las configuraciones evaluadas, 8 hilos ofrecen el mayor speedup. Si se busca un mejor equilibrio entre speedup y eficiencia, los resultados indican que 4 hilos son una opción más conveniente.

Como trabajo futuro se recomienda investigar formas de conservar la vectorización del ciclo interno bajo OpenMP, ya que recuperar ese factor duplicaría el rendimiento de todas las configuraciones paralelas sin necesidad de utilizar más hilos.

## Referencias

- [1] OpenMP Architecture Review Board. *OpenMP Application Programming Interface Version 5.2*. 2021. <https://www.openmp.org/specifications/>
- [2] Ian Foster. *Designing and Building Parallel Programs*. Addison-Wesley. 1995.
- [3] Sverre J. Aarseth. *Gravitational N-Body Simulations: Tools and Algorithms*. Cambridge University Press. 2003.
- [4] Simple DirectMedia Layer. *SDL2 Documentation*. <https://wiki.libsdl.org/SDL2/FrontPage>

## A. Anexo 1. Diagrama de flujo del programa

El diagrama resume la captura de argumentos, la validación de entradas, la configuración de OpenMP, la reserva de recursos, las secciones paralelas, la sincronización, el renderizado, el

despliegue de resultados y la liberación de recursos.



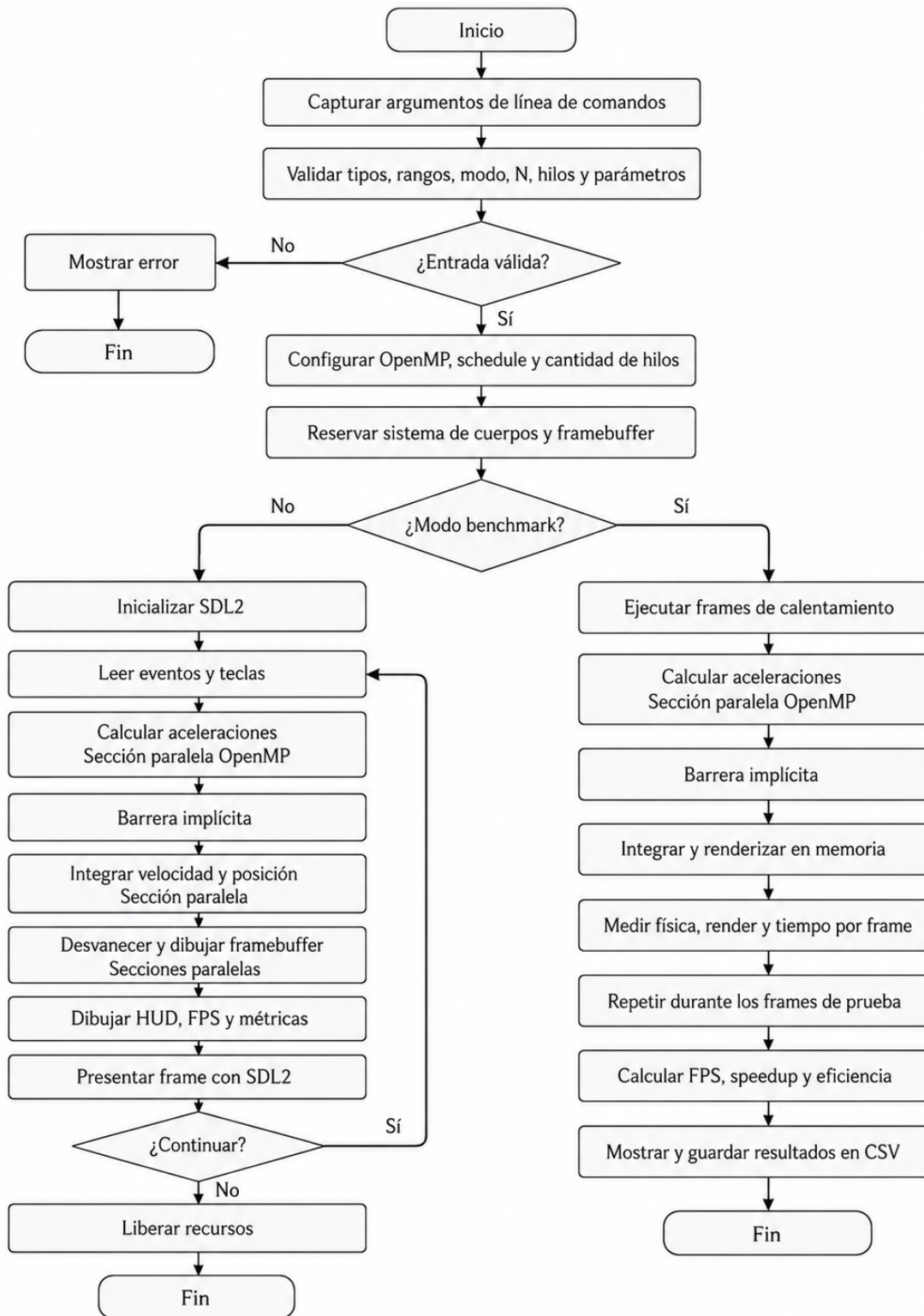


Figura 5: Diagrama de flujo general del programa

## B. Anexo 2. Catálogo de funciones

Para cada función se indican los parámetros de entrada con su nombre y tipo exactos, el tipo de retorno y el propósito de la rutina. Las funciones miembro de `SistemaNCuerpos` no reciben parámetros: operan sobre los arreglos internos del objeto, por lo que se indica cuáles arreglos leen y cuáles escriben.

| Función o rutina                                     | Entradas (nombre y tipo)                                                                                              | Salida                   | Descripción                                                                                                                           |
|------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>cli.cpp / cli.h — argumentos y validación</b>     |                                                                                                                       |                          |                                                                                                                                       |
| <code>a_long</code>                                  | <code>const std::string&amp; texto, long&amp; out</code>                                                              | <code>bool</code>        | Convierte <code>texto</code> a entero largo. Devuelve falso si el texto está vacío, si sobra basura al final o si hay desbordamiento. |
| <code>a_double</code>                                | <code>const std::string&amp; texto, double&amp; out</code>                                                            | <code>bool</code>        | Convierte <code>texto</code> a un valor real y comprueba errores de conversión.                                                       |
| <code>siguiente_valor</code>                         | <code>int argc, char** argv, int&amp; i, std::string&amp; out</code>                                                  | <code>bool</code>        | Obtiene el valor que sigue a una bandera. Devuelve falso si la bandera venía al final de la línea sin valor.                          |
| <code>leer_entero</code>                             | <code>int argc, char** argv, int&amp; i, const std::string&amp; bandera, long lo, long hi, long&amp; out</code>       | <code>bool</code>        | Lee un entero y comprueba que se encuentre en el intervalo cerrado <code>[lo,hi]</code> .                                             |
| <code>leer_real</code>                               | <code>int argc, char** argv, int&amp; i, const std::string&amp; bandera, double lo, double hi, double&amp; out</code> | <code>bool</code>        | Lee un valor real y comprueba que se encuentre en el intervalo <code>(lo,hi]</code> , abierto por abajo.                              |
| <code>reparto_desde_texto</code>                     | <code>const std::string&amp; texto, Reparto&amp; out</code>                                                           | <code>bool</code>        | Convierte <code>static</code> , <code>dynamic</code> o <code>guided</code> al tipo interno <code>Reparto</code> .                     |
| <code>texto_de_reparto</code>                        | <code>Reparto r</code>                                                                                                | <code>const char*</code> | Devuelve el nombre de la política de reparto, para el CSV y los mensajes de error.                                                    |
| <code>imprimir_ayuda</code>                          | <code>const char* nombre_programa</code>                                                                              | <code>void</code>        | Muestra las banderas disponibles, sus rangos, las teclas y ejemplos de uso.                                                           |
| <code>parsear_argumentos</code>                      | <code>int argc, char** argv, Parametros&amp; out</code>                                                               | <code>bool</code>        | Procesa y valida toda la configuración de la línea de comandos. Devuelve falso ante cualquier entrada inválida.                       |
| <b>nbody.cpp / nbody.h — física de la simulación</b> |                                                                                                                       |                          |                                                                                                                                       |
| <code>modo_desde_texto</code>                        | <code>const std::string&amp; texto, Modo&amp; out</code>                                                              | <code>bool</code>        | Convierte <code>colision</code> , <code>galaxia</code> o <code>nube</code> al tipo interno <code>Modo</code> .                        |

| Función o rutina                                        | Entradas (nombre y tipo)                                                                                                                                                                 | Salida                                                           | Descripción                                                                                                                               |
|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| texto_de_modo                                           | Modo m                                                                                                                                                                                   | const char*                                                      | Devuelve el nombre del escenario actual.                                                                                                  |
| SistemaNCuerpos<br>(constructor)                        | int n, int ancho, int alto, Modo<br>modo, uint32_t semilla, const<br>ParametrosFisica& fisica                                                                                            | Objeto construido                                                | Reserva los ocho arreglos internos de tamaño n y siembra el escenario. Lanza <code>std::bad_alloc</code> si n no cabe en memoria.         |
| sembrar_disco                                           | Xorshift32& r, int desde, int cuenta,<br>float cx, float cy, float vx_bulk, float<br>vy_bulk, int giro                                                                                   | void                                                             | Crea un disco galáctico con núcleo masivo en el índice <code>desde</code> y estrellas en órbita circular. <code>giro</code> vale +1 o -1. |
| sembrar_nube                                            | Xorshift32& r                                                                                                                                                                            | void                                                             | Distribuye los cuerpos de forma pseudoaleatoria con masas iguales y velocidades iniciales pequeñas.                                       |
| reiniciar                                               | Modo modo, uint32_t semilla                                                                                                                                                              | void                                                             | Vuelve a sembrar el escenario sin reasignar los arreglos.                                                                                 |
| calcular_aceleraciones                                  | Sin parámetros. Lee <code>px_</code> , <code>py_</code> , <code>masa_</code><br>( <code>std::vector&lt;float&gt;</code> ); escribe <code>ax_</code> , <code>ay_</code>                   | void                                                             | Calcula la aceleración gravitacional de cada cuerpo. El ciclo externo se paraleliza con <code>schedule(runtime)</code> .                  |
| integrar                                                | Sin parámetros. Lee <code>ax_</code> , <code>ay_</code> ; escribe <code>vx_</code> ,<br><code>vy_</code> , <code>px_</code> , <code>py_</code> ( <code>std::vector&lt;float&gt;</code> ) | void                                                             | Actualiza velocidad y posición mediante Euler semiimplícito, tras la barrera implícita.                                                   |
| energia_cinetica                                        | Sin parámetros. Lee <code>masa_</code> , <code>vx_</code> , <code>vy_</code><br>( <code>std::vector&lt;float&gt;</code> )                                                                | double                                                           | Calcula la energía cinética total usando <code>reduction(+:ke)</code> .                                                                   |
| energia_potencial                                       | Sin parámetros. Lee <code>masa_</code> , <code>px_</code> , <code>py_</code><br>( <code>std::vector&lt;float&gt;</code> )                                                                | double                                                           | Calcula la energía potencial de todos los pares con <code>reduction(+:pe)</code> y <code>schedule(dynamic)</code> .                       |
| n / rapidez_ref /<br>px / py / vx / vy /<br>tono_jitter | Sin parámetros                                                                                                                                                                           | int, float y const<br><code>std::vector&lt;float&gt;&amp;</code> | Acceso de solo lectura al tamaño, la escala de color y los arreglos internos, para el renderizado.                                        |
| <b>rng.h — generador pseudoaleatorio</b>                |                                                                                                                                                                                          |                                                                  |                                                                                                                                           |
| Xorshift32<br>(constructor)                             | uint32_t semilla                                                                                                                                                                         | Objeto construido                                                | Inicializa el generador y evita el estado cero, que se quedaría bloqueado.                                                                |
| siguiente                                               | Sin parámetros. Usa estado (uint32_t)                                                                                                                                                    | uint32_t                                                         | Produce el siguiente valor pseudoaleatorio y actualiza el estado privado.                                                                 |
| uniforme01                                              | Sin parámetros                                                                                                                                                                           | float                                                            | Produce un valor pseudoaleatorio en el intervalo [0, 1).                                                                                  |

| Función o rutina                           | Entradas (nombre y tipo)                                                                                              | Salida            | Descripción                                                                                                                                                                             |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rango                                      | float lo, float hi                                                                                                    | float             | Produce un valor pseudoaleatorio en [lo,hi).                                                                                                                                            |
| <b>palette.cpp — color</b>                 |                                                                                                                       |                   |                                                                                                                                                                                         |
| mezclar                                    | uint8_t a, uint8_t b, float f                                                                                         | uint8_t           | Interpola linealmente entre dos valores de un canal de color.                                                                                                                           |
| color_por_rapidez                          | float t (rapidez normalizada, se recorta a [0,1])                                                                     | ColorRGB          | Obtiene un color de la rampa de cinco paradas: azul, cian, blanco, amarillo y naranja.                                                                                                  |
| <b>render.cpp / render.h — dibujo</b>      |                                                                                                                       |                   |                                                                                                                                                                                         |
| sumar_saturando                            | uint32_t dst, int r, int g, int b                                                                                     | uint32_t          | Suma brillo a un píxel ARGB8888 sin permitir que un canal exceda 255.                                                                                                                   |
| NucleoDestello (constructor)               | Sin parámetros                                                                                                        | Objeto construido | Precalcula los 25 pesos del destello, una sola vez al arrancar el programa.                                                                                                             |
| desvanecer                                 | std::vector<uint32_t>& fb, uint32_t factor                                                                            | void              | Multiplica cada canal por factor/256 para generar las estelas. Ciclo paralelo por píxel.                                                                                                |
| limpiar                                    | std::vector<uint32_t>& fb, uint32_t color                                                                             | void              | Rellena todo el framebuffer con un solo color.                                                                                                                                          |
| dibujar_cuerpos                            | std::vector<uint32_t>& fb, int ancho, int alto, const SistemaNCuerpos& sistema, const std::vector<float>& tono_jitter | void              | Dibuja cada cuerpo como un destello aditivo coloreado por rapidez más el desplazamiento de tono. Divide el framebuffer en bandas de filas disjuntas para evitar condiciones de carrera. |
| <b>hud.cpp / hud.h — panel de métricas</b> |                                                                                                                       |                   |                                                                                                                                                                                         |
| empacar                                    | uint8_t r, uint8_t g, uint8_t b                                                                                       | uint32_t          | Convierte componentes RGB al formato ARGB8888.                                                                                                                                          |
| dibujar_texto                              | std::vector<uint32_t>& fb, int ancho, int alto, int x, int y, const char* texto, uint32_t color                       | void              | Dibuja una línea de texto con la fuente bitmap 5x7 incluida en el programa.                                                                                                             |
| atenuar_recuadro                           | std::vector<uint32_t>& fb, int ancho, int alto, int x0, int y0, int w, int h                                          | void              | Oscurece el fondo del panel para que el texto se lea sobre cualquier escena.                                                                                                            |
| dibujar_hud                                | std::vector<uint32_t>& fb, int ancho, int alto, const MetricasHud& m                                                  | void              | Dibuja el panel de métricas en la esquina superior izquierda. La línea de FPS se pinta verde si alcanza 30 y roja si no.                                                                |

| Función o rutina                                      | Entradas (nombre y tipo)                                                                                      | Salida            | Descripción                                                                                                            |
|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|-------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>main.cpp — ventana, ciclo principal y medición</b> |                                                                                                               |                   |                                                                                                                        |
| <code>hilos_activos</code>                            | Sin parámetros                                                                                                | <code>int</code>  | Devuelve la cantidad de hilos configurados en OpenMP, o 1 en la versión secuencial.                                    |
| <code>fijar_hilos</code>                              | <code>int n</code>                                                                                            | <code>void</code> | Establece la cantidad de hilos. Un valor menor o igual a cero deja que OpenMP decida.                                  |
| <code>ContextoSDL</code><br>(destructor)              | Sin parámetros. Opera sobre <code>SDL_Window*</code> , <code>SDL_Renderer*</code> y <code>SDL_Texture*</code> | <code>void</code> | Destruye textura, renderer y ventana, comprobando cada puntero. Cubre también los fallos a mitad de la inicialización. |
| <code>iniciar_sdl</code>                              | <code>ContextoSDL&amp; ctx, const Parametros&amp; p</code>                                                    | <code>bool</code> | Inicializa SDL2 y crea ventana, renderer y textura. Cae a renderer por software si no hay aceleración disponible.      |
| <code>aplicar_reparto</code>                          | <code>Reparto r, int trozo</code>                                                                             | <code>void</code> | Configura la política que utilizará <code>schedule(runtime)</code> . Un trozo de cero deja que OpenMP elija.           |
| <code>ejecutar_benchmark</code>                       | <code>const Parametros&amp; p</code>                                                                          | <code>int</code>  | Ejecuta el calentamiento, mide física y renderizado por separado y emite una fila CSV. No inicializa SDL2.             |
| <code>main</code>                                     | <code>int argc, char** argv</code>                                                                            | <code>int</code>  | Coordina validación, configuración, simulación, eventos, métricas y liberación de recursos.                            |

## C. Anexo 3. Bitácora de pruebas

La bitácora final se generó mediante el script `scripts/bench.ps1`. Se utilizaron 10 repeticiones por configuración y 20 frames medidos por repetición. Se probaron 4000 y 8000 cuerpos con la versión secuencial y con la versión paralela utilizando 1, 2, 4 y 8 hilos. El archivo `bench.csv` contiene un total de 100 mediciones individuales.

El comando utilizado para ejecutar la prueba fue:

```
.\scripts\bench.ps1 -Repeticiones 10 -Frames 20  
-ListaN 4000,8000 -ListaHilos 1,2,4,8
```

Cuadro 5: Resumen de la bitácora de pruebas

| N    | Versión    | Hilos | Reps. | ms/frame | FPS   | Speedup | Eficiencia |
|------|------------|-------|-------|----------|-------|---------|------------|
| 4000 | Secuencial | 1     | 10    | 15.149   | 66.2  | 1.00x   | –          |
| 4000 | Paralela   | 1     | 10    | 29.892   | 33.5  | 0.50x   | 50 %       |
| 4000 | Paralela   | 2     | 10    | 16.104   | 62.4  | 0.93x   | 46 %       |
| 4000 | Paralela   | 4     | 10    | 9.855    | 101.9 | 1.52x   | 38 %       |
| 4000 | Paralela   | 8     | 10    | 8.519    | 117.5 | 1.75x   | 22 %       |
| 8000 | Secuencial | 1     | 10    | 58.745   | 17.0  | 1.00x   | –          |
| 8000 | Paralela   | 1     | 10    | 131.675  | 7.7   | 0.45x   | 45 %       |
| 8000 | Paralela   | 2     | 10    | 71.746   | 14.0  | 0.82x   | 41 %       |
| 8000 | Paralela   | 4     | 10    | 44.808   | 22.6  | 1.33x   | 33 %       |
| 8000 | Paralela   | 8     | 10    | 42.649   | 23.6  | 1.38x   | 17 %       |

La tabla resume los promedios obtenidos a partir de las 10 repeticiones de cada configuración. La eficiencia solamente se reporta para la versión paralela, ya que expresa qué proporción del speedup se obtiene por cada hilo utilizado. El speedup de las filas secuenciales es 1.00x por definición, al ser la referencia de comparación.

La captura muestra el resumen producido al finalizar el script. En ella se observan el tiempo promedio por frame, los FPS, el speedup y la eficiencia de cada configuración evaluada.

```

PS C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VIII SEMESTRE\Computación Paralela\Computacion_Paralela> .\scripts\bench.ps1 -Repeticiones 10 -Frames 20 -Lista N 4000,8000 -ListaHilos 1,2,4,8
Escrito resultados/bench.csv (100 mediciones)

```

| N    | hilos | ms/frame | FPS   | speedup | eficiencia |
|------|-------|----------|-------|---------|------------|
| 4000 | seq   | 15.149   | 66.2  | 0.99x   | --         |
| 4000 | 1     | 29.892   | 33.5  | 0.50x   | 50 %       |
| 4000 | 2     | 16.104   | 62.4  | 0.93x   | 46 %       |
| 4000 | 4     | 9.855    | 101.9 | 1.52x   | 38 %       |
| 4000 | 8     | 8.519    | 117.5 | 1.75x   | 22 %       |
| 8000 | seq   | 58.745   | 17.0  | 1.00x   | --         |
| 8000 | 1     | 131.675  | 7.7   | 0.45x   | 45 %       |
| 8000 | 2     | 71.746   | 14.0  | 0.82x   | 41 %       |
| 8000 | 4     | 44.808   | 22.6  | 1.33x   | 33 %       |
| 8000 | 8     | 42.649   | 23.6  | 1.38x   | 17 %       |

```

PS C:\Users\ninan\OneDrive\Documentos\Escritorio\UVG\VIII SEMESTRE\Computación Paralela\Computacion_Paralela> 

```

Figura 6: Resumen de las 100 mediciones generadas por el benchmark